

Unit 3: Advanced Server Side Scripting

Object Oriented Programming in PHP: Classes and Objects, Defining and Using properties and methods, Constructors and Destructors, Method Overriding, Encapsulation, Inheritance, Polymorphism, Static Members, Exception Handling

AJAX(Asynchronous JavaScript and XML): Using PHP, Using PHP +MySQL

jQuery: Playing With Elements, Hiding and Unhiding Images. jQuery UI

JOOMLA: Introduction To CMS, Installation, Handling Joomla Back End.. Customization In Joomla, Introduction To Extensions, Installation and Uses Of Extensions in Joomla, Template Development In Joomla, Artiseer(IDE), Module Development In Joomla, Component Development In Joomla, Introduction To MVC(Model, View and Controller)

WordPress Administrator Level: Theme Integration, Creating pages, Managing Posts, Managing Widgets

Object Oriented Programming in PHP

- OOP

- In OOP, there's a concept of "class", which is used to model or map a real-world entity to a template of data (properties) and functionality (methods).
- An "object" is an instance of a class, and you can create multiple instances of the same class. For example, there is a single Person class, but many person objects can be instances of this class—dan, zainab, hector, etc.
- The class defines properties. For example, for the Person class, we might have name, age, and phoneNumber. Then each person object will have its own values for those properties.

Creating Class and Objects (Cont...)

- PHP Class

- A class is a template which represents a real-world entity, and it defines properties and methods of the entity.

```
<?php
class Employee
{
    private $first_name;
    private $last_name;
    private $age;
    public function __construct($first_name,
    $last_name, $age)
    {
        $this->first_name = $first_name;
        $this->last_name = $last_name;
        $this->age = $age;
    }
    public function getFirstName()
    {
```

```
        return $this->first_name;
    }
    public function getLastName()
    {
        return $this->last_name;
    }
    public function getAge()
    {
        return $this->age;
    }
}
?>
```

- The class Employee statement in the first line defines the Employee class. Then, we go on to declare the properties, the constructor, and the other class methods.

Creating Class and Objects (Cont...)

- Class Properties in PHP

- We could think of class properties as variables that are used to hold information about the object.
- In the above example, we've defined three properties—first_name, last_name, and age.
- In most cases, class properties are accessed via instantiated objects.
- These properties are private, which means they can only be accessed from within the class.

- Constructors for PHP Classes

- A constructor is a special class method which is called automatically when we instantiate an object.
- A constructor is used to initialize object properties when the object is being created.
- We can define a constructor by defining the `__construct` method.

Creating Class and Objects (Cont...)

- Methods for PHP Classes
 - We can think of class methods as functions that perform specific actions associated with objects. In most cases, they are used to access and manipulate object properties and perform related operations.
 - In the above example, we've defined the `getLastName` method, which returns the last name associated with the object.

Creating Class and Objects (Cont...)

- Object in PHP

- when we want to use a class, we need to instantiate it, and the end result is an object.
- So we could think of a class as a blueprint, and an object is an actual thing that you can work with.
- In the context of the Employee class which we've just created in the previous section, let's see how to instantiate an object of that class.

```
<?php
$objEmployee = new Employee('Bob', 'Smith', 30);
echo $objEmployee->getFirstName(); // print 'Bob'
echo $objEmployee->getLastName(); // prints 'Smith'
echo $objEmployee->getAge(); // prints '30'
?>
```

- need to use the new keyword when we want to instantiate an object of any class along with its class name, and we'll get back a new object instance of that class.

Creating Class and Objects (Cont...)

- Object in PHP

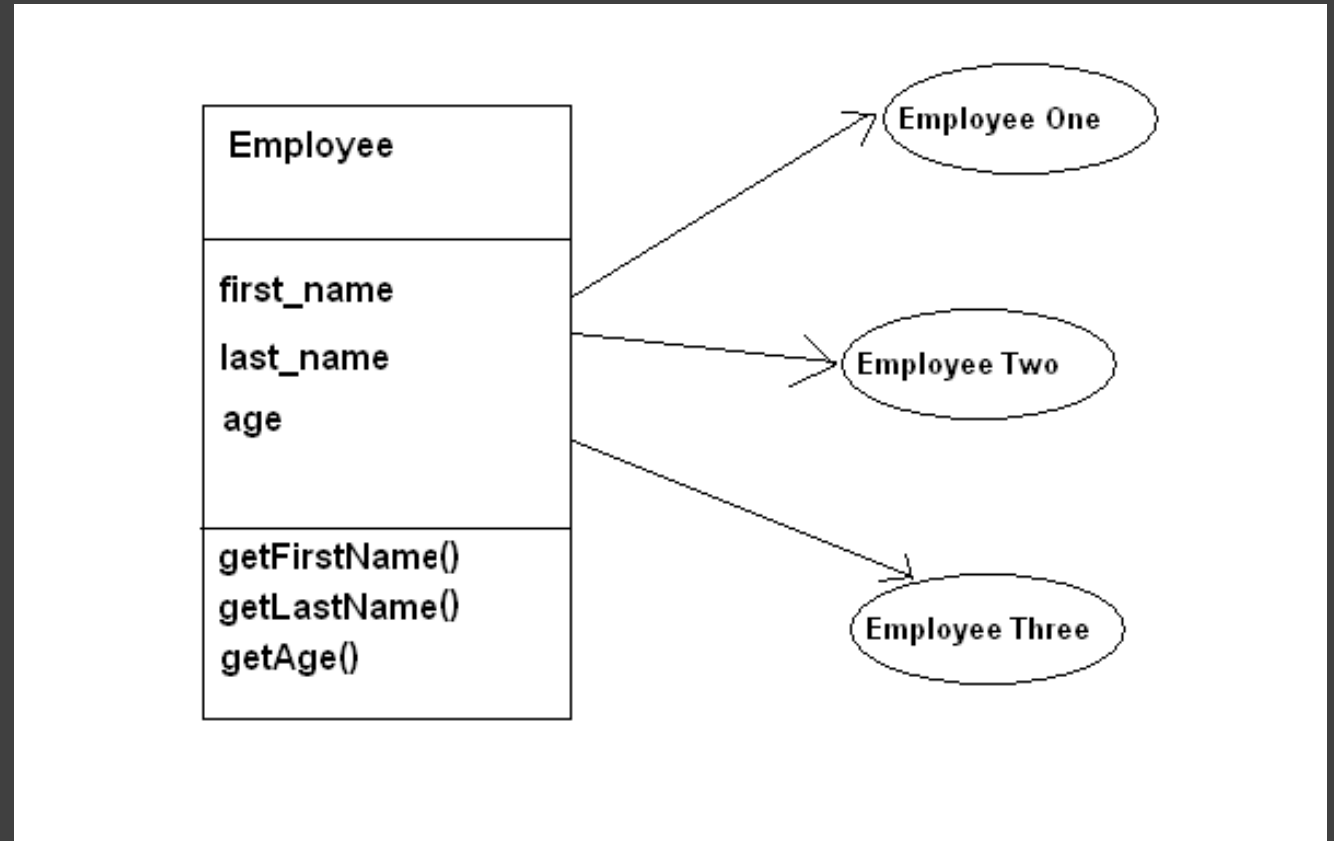
- If a class has defined the `__construct` method and it requires arguments, we need to pass those arguments when we instantiate an object.
- In our case, the `Employee` class constructor requires three arguments, and thus we've passed these when we created the `$objEmployee` object.
- The `__construct` method is called automatically when the object is instantiated.
- We've called class methods on the `$objEmployee` object to print the information which was initialized during object creation.
- We can create multiple objects of the same class, as shown in the following snippet.

```
<?php
$objEmployeeOne = new Employee('Bob', 'Smith', 30);
    echo $objEmployeeOne->getFirstName(); // prints 'Bob'
    echo $objEmployeeOne->getLastName(); // prints 'Smith'
    echo $objEmployeeOne->getAge(); // prints '30'
$objEmployeeTwo = new Employee('John', 'Smith', 34);
    echo $objEmployeeTwo->getFirstName(); // prints 'John'
    echo $objEmployeeTwo->getLastName(); // prints 'Smith'
    echo $objEmployeeTwo->getAge(); // prints '34'
```

```
?>
```

Creating Class and Objects (Cont...)

- Object in PHP
 - The following image is a graphical representation of the Employee class and some of its instances.
 - A class is a blueprint which we can use to create structured objects.



Constructors and Destructors

- Constructors are special member functions for initial settings of newly created object instances from a class, which is the key part of the object-oriented concept in PHP5.
- Constructors are the very basic building blocks that define the future object and its nature. You can say that the Constructors are the blueprints for object creation providing values for member functions and member variables.
- If the PHP Class has a constructor, then at the time of object creation, the constructor of the class is called. The constructors have no Return Type, so they do not return anything not even void.
- Once the object is initialized, the constructor is automatically called. Destructors are for destroying objects and automatically called at the end of execution.
- A destructor is called when the object is destructed or the script is stopped or exited.
- If we create a `__destruct()` function, PHP will automatically call this function at the end of the script.

Constructors and Destructors

Syntax:

- `__construct()`:

```
function __construct()  
{  
    // initialize the object and its properties by assigning  
    // values  
}
```

- `__destruct()`:

```
function __destruct()  
{  
    // destroying the object or clean up resources here  
}
```

Note: The constructor is defined in the public section of the Class. Even the values to properties of the class are set by Constructors.

Constructors and Destructors

- Constructor types:
 - Default Constructor: It has no parameters, but the values to the default constructor can be passed dynamically.
 - Parameterized Constructor: It takes the parameters, and also you can pass different values to the data members.
 - Copy Constructor: It accepts the address of the other objects as a parameter.
- Default Constructor: By using function `__construct()`, We can define a constructor.

- Example:

```
<?PHP
class Tree
{
    function Tree()
    {
        echo "Its a User-defined Constructor of the class Tree";
    }

    function __construct()
    {
        echo "Its a Pre-defined Constructor of the class Tree";
    }
}
```

```
$obj= new Tree();
```

```
?>
```

```
//output: Its a Pre-defined Constructor of the class Tree
```

Note: In the case of Pre-defined Constructor(`__construct`) and user-defined constructor in the same class, the Pre-defined Constructor becomes Constructor while user-defined constructor becomes the normal method.

Constructors and Destructors

- Parameterized Constructor:

- The constructor of the class accepts arguments or parameters.
- The -> operator is used to set value for the variables. In the constructor method, you can assign values to the variables during object creation.
- Constructors start with two underscores and generally look like normal PHP functions. Sometimes these constructors are called as magic functions starting with two underscores and with some extra functionality than normal methods.
- After creating an object of some class that includes constructor, the content of constructor will be automatically executed.

- Example:

```
<?php
class Employee
{
    Public $name;
    Public $position;
    function __construct($name,$position)
    {
        // This is initializing the class properties
```

```
        $this->name=$name;
        $this->position=$position;
    }
    function show_details()
    {
        echo $this->name." : ";
        echo "Your position is ".$this->profile."\n";
    }
}
$employee_obj= new Employee("Rakesh","developer");
$employee_obj->show_details();
$employee2= new Employee("Vikas","Manager");
$employee2->show_details();

?>
```

output:

Rakesh : Your position is developer
Vikas : Your position is Manager

Constructors and Destructors

- Advantages of using Constructors:
 - Constructors provides the ability to pass parameters which are helpful in automatic initialization of the member variables during creation time .
 - The Constructors can have as many parameters as required and they can be defined with the default arguments.
 - They encourage re-usability avoiding re-initializing whenever instance of the class is created .
 - We can start session in constructor method so that you don't have to start in all the functions everytime.
 - They can call class member methods and functions.
 - They can call other Constructors even from Parent class.

Constructors and Destructors

- **Destructor:**
 - Destructor is also a special member function which is exactly the reverse of constructor method and is called when an instance of the class is deleted from the memory.
 - Destructors (`__destruct ( void): void`) are methods which are called when there is no reference to any object of the class or goes out of scope or about to release explicitly.
 - They don't have any types or return value. It is just called before de-allocating memory for an object or during the finish of execution of PHP scripts or as soon as the execution control leaves the block.
 - Global objects are destroyed when the full script or code terminates. Cleaning up of resources before memory release or closing of files takes place in the destructor method, whenever they are no longer needed in the code. The automatic destruction of class objects is handled by PHP Garbage Collector.

Constructors and Destructors

Example:

```
<?php
class myclass{
    function __construct(){
        echo "object is initialized
";
    }
    function __destruct(){
        echo "object is destroyed
";
    }
}
```

```
$obj=new myclass();
?>
```

This will show following result:
object is initialized
object is destroyed

Note: The destructor method is called when the PHP code is executed completely by its last line by using PHP exit() or die() functions.

Constructors and Destructors

- Advantages of destructors:
 - Destructors give chance to objects to free up memory allocation , so that enough space is available for new objects or free up resources for other tasks.
 - It effectively makes programs run more efficiently and are very useful as they carry out clean up tasks.

Method Overriding

- Method overriding allows a child class to provide a specific implementation of a method already provided by its parent class.
- To override a method, we redefine that method in the child class with the same name, parameters, and return type.
- The method in the parent class is called overridden method, while the method in the child class is known as the overriding method. The code in the overriding method overrides (or replaces) the code in the overridden method.
- PHP will decide which method (overridden or overriding method) to call based on the object used to invoke the method.
- If an object of the parent class invokes the method, PHP will execute the overridden method.
- But if an object of the child class invokes the method, PHP will execute the overriding method.

Method Overriding

```
<?php
// PHP program to implement
// function overriding
// This is parent class
class P {
    // Function geeks of parent class
    function geeks() {
        echo "Parent";
    }
}
// This is child class
class C extends P {
    // Overriding geeks method
```

```
function geeks() {
    echo "\nChild";
}
// Reference type of parent
$p = new P;
// Reference type of child
$c= new C;
// print Parent
$p->geeks();
// Print child
$c->geeks();
?>
```

Encapsulation

- OOPs concept of Encapsulation in PHP means, enclosing the internal details of the object to protect from external sources.
- It describes, combining the class, data variables and member functions that work on data together within a single unit to form an object. Otherwise, its the bundling of properties and behavior in a single class unit.
- Data is not accessed directly, in fact, they are accessed through functions(GET or SET) written inside the class. Attributes are kept private but getter(GET) and setter(SET) methods are kept public for manipulation of these attributes.
- Data encapsulation led to the important concept of data hiding in object-oriented programming also known as Data abstraction. Data abstraction is a mechanism of exposing only the interfaces and hiding the implementation details from the user.
- Data Encapsulation steps:
 - Declare each property private.
 - Create public set method for each property to set the values of properties.
 - Create public get method for each property to get the values of properties.

Encapsulation

- Example:

```
<?php
class person {
    private $Name;
    private $Age;
    //method to set the value of Name
    public function setName($name) {
        $this->Name = $name;
    }
    //method to get the value of Name
    public function getName() {
        return $this->Name;
    }
    //method to set the value of Age
    public function setAge($age) {
        $this->Age = $age;
    }
}
```

```
//method to get the value of Age
public function getAge() {
    return $this->Age;
}
};
```

```
$p1 = new person();
```

```
//set the name and age
$p1->setName("John");
$p1->setAge(25);
//get the name and age
echo $p1->getName()." is ".$p1->getAge()."
years old.";
?>
```

Output: John is 25 years old.

Inheritance (cont...)

- Example 1:

```
<?php
class a
{
    function fun1()
    {
        echo "java";
    }
}
class b extends a
{
    function fun2()
    {
```

```
        echo "bca";
    }
}
/* Create object (child class instance) */
$obj= new b();
$obj->fun1(); /*call member function
of ParentClass*/
?>
```

Inheritance (cont...)

- Example 2:

```
<?php
/* Parent Class (also called superclass and
base class)*/
class ParentClass
{
    function a()
    {
        echo "A member function of the base
class.";
    }
}
/* Child class (also called Derived class,
extended class and subclass)*/
class ChildClass extends ParentClass
{
    function b()
    {
```

```
        echo "A member function of the child
class.";
    }
}
```

```
$Obj = new ChildClass(); /* Create object
(child class instance) */
$Obj->a(); /*call member function of
ParentClass*/
$Obj->b(); /*call member function of
ChildClass*/
?>
```

Output:

A member function of the base class.
A member function of the child class.

Polymorphism

- Syntax:

```
Class a{  
  //methods  
  Public function b(){  
    //methods  
  }  
}  
Class b extends a{  
  //methods  
  Public function b(){  
    //methods which are in/not in the function of the class a.  
  }  
}
```

Polymorphism (cont...)

- Example 2:

```
<?php
class base11
{
    function add1($a1,$b1)
    {
        $res1=$a1*$b1;
        echo "Multiplication = ".$res1;
    }
}
class child1 extends base11
{
    function add1($a1,$b1)
    {
```

```
        $res1=$a1+$b1;
        echo "Sum = ".$res1;
    }
}
$obj= new child1();
$obj->add1(1000,500);
?>
```

Output:
Sum = 1500

Static Members

- The static keyword is used to declare properties and methods of a class as static. Static properties and methods can be used without creating an instance of the class.
- The static keyword is also used to declare variables in a function which keep their value after the function has ended.
- PHP - Static Methods:
 - Static methods can be called directly - without creating an instance of the class first.
 - Static methods are declared with the static keyword.
 - Syntax:

```
<?php
class ClassName {
    public static function staticMethod() {
        echo "Hello World!";
    }
}
?>
```
 - To access a static method use the class name, double colon (::), and the method name:
 - Syntax:

```
ClassName::staticMethod();
```

Static Members (cont...)

- PHP - Static Methods:

- Example 1:

```
<?php
class greeting {
    public static function welcome() {
        echo "Hello World!";
    }
}
```

```
// Call static method
greeting::welcome();
?>
```

Static Members (cont...)

- PHP - Static Methods:

- A class can have both static and non-static methods. A static method can be accessed from a method in the same class using the self keyword and double colon (::).

- Example 2:

```
<?php
class greeting {
    public static function welcome() {
        echo "Hello World!";
    }

    public function __construct() {
        self::welcome();
    }
}

new greeting();
?>
```

Static Members (cont...)

- PHP - Static Methods:

- Static methods can also be called from methods in other classes. To do this, the static method should be public.

- Example 3:

```
<?php
class A {
    public static function welcome() {
        echo "Hello World!";
    }
}
```

```
class B {
    public function message() {
        A::welcome();
    }
}
```

```
$obj = new B();
echo $obj->message();
?>
```

Exception Handling

- Exceptions are used to change the normal flow of a script if a specified error occurs.
- With PHP 5 came a new object oriented way of dealing with errors.
- Exception handling is used to change the normal flow of the code execution if a specified error (exceptional) condition occurs. This condition is called an exception.
- An exception is an unexpected program result that can be handled by the program itself.
- Exception Handling in PHP is almost similar to exception handling in all programming languages.
- This is what normally happens when an exception is triggered:
 - The current code state is saved
 - The code execution will switch to a predefined (custom) exception handler function
 - Depending on the situation, the handler may then resume the execution from the saved code state, terminate the script execution or continue the script from a different location in the code

Exception Handling (cont...)

- We will show different error handling methods:
 - Basic use of Exceptions
 - Creating a custom exception handler
 - Multiple exceptions
 - Re-throwing an exception
 - Setting a top level exception handler
- Note: Exceptions should only be used with error conditions, and should not be used to jump to another place in the code at a specified point.
- PHP provides the following specialized keywords for this purpose.
 - **try**: It represents a block of code in which exceptions can arise.
 - **catch**: It represents a block of code that will be executed when a particular exception has been thrown.
 - **throw**: It is used to throw an exception. It is also used to list the exceptions that a function throws, but doesn't handle itself.
 - **finally**: It is used in place of a catch block or after a catch block basically it is put for cleanup activity in PHP code.

Exception Handling (cont...)

- Example 1

```
<?php
//create function with an exception
function checkNum($number) {
    if($number>1) {
        throw new Exception("Value must be 1
or below");
    }
    return true;
}
//trigger exception in a "try" block
try {
    checkNum(2);
```

```
//If the exception is thrown, this text
will not be shown
```

```
    echo 'If you see this, the number is 1 or
below';
}
```

```
//catch exception
catch(Exception $e) {
    echo 'Message: ' . $e->getMessage();
}
?>
```

Output:

Message: Value must be 1 or below

Exception Handling (cont...)

- Example1 explained:
 - The code above throws an exception and catches it:
 - The checkNum() function is created. It checks if a number is greater than 1. If it is, an exception is thrown
 - The checkNum() function is called in a "try" block
 - The exception within the checkNum() function is thrown
 - The "catch" block retrieves the exception and creates an object (\$e) containing the exception information
 - The error message from the exception is echoed by calling \$e->getMessage() from the exception object
 - However, one way to get around the "every throw must have a catch" rule is to set a top level exception handler to handle errors that slip through.

Ajax using PHP and MYSQL

- AJAX can be used for interactive communication with a database using php.
- The database table we use in the example above looks like this:

id	Firstname	LastName	Age	Hometown
4	Ram	Sharma	22	KTM
5	Shyam	hamal	33	BKT

Ajax using PHP and MYSQL (cont..)

- HTML Code(index.html)

```
<html>
<head>
<script>
function showUser(str) {
  if (str == "") {
    document.getElementById("txtHint").innerHTML = "";
    return;
  } else {
    var xmlhttp = new XMLHttpRequest();
    xmlhttp.onreadystatechange = function() {
      if (this.readyState == 4 && this.status == 200) {
        document.getElementById("txtHint").innerHTML =
this.responseText;
      }
    };
    xmlhttp.open("GET","get.php?q="+str,true);
    xmlhttp.send();
  }
}
</script>
</head>
<body>
```

```
<form>
<select name="users" onchange="showUser(this.value)">
  <option value="">Select a person:</option>
  <option value="4">Ram</option>
  <option value="5">Shyam</option>
</select>
</form>
<br>
<div id="txtHint"><b>Person info will be listed here...</b></div>
</body>
```

- In the example, when a user selects a person in the dropdown list above, a function called "showUser()" is executed.
- The function is triggered by the onchange event.

Ajax using PHP and MYSQL (cont..)

- First, check if person is selected. If no person is selected (`str == ""`), clear the content of `txtHint` and exit the function. If a person is selected, do the following:
 - Create an `XMLHttpRequest` object
 - Create the function to be executed when the server response is ready
 - Send the request off to a file on the server
 - Notice that a parameter (`q`) is added to the URL (with the content of the dropdown list)

Ajax using PHP and MYSQL (cont..)

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<?php
$q = intval($_GET['q']);
$con = mysqli_connect('localhost','root','');
if (!$con) {
    die('Could not connect: ' . mysqli_error($con));
}
mysqli_select_db($con,"ajax_demo");
$sql="SELECT * FROM admin WHERE id = ".$q."";
$result = mysqli_query($con,$sql);
echo "<table>
<tr>
<th>Firstname</th>
<th>Lastname</th>
```

```
<th>Age</th>
<th>Hometown</th>
</tr>";
while($row = mysqli_fetch_array($result)) {
    echo "<tr>";
    echo "<td>" . $row['Firstname'] . "</td>";
    echo "<td>" . $row['LastName'] . "</td>";
    echo "<td>" . $row['Age'] . "</td>";
    echo "<td>" . $row['Hometown'] . "</td>";
    echo "</tr>";
}
echo "</table>";
mysqli_close($con);
?>
</body>
</html>
```

Ajax using PHP and MYSQL (cont..)

- The page on the server called by the JavaScript above is a PHP file called "get.php".
- The source code in "get.php" runs a query against a MySQL database, and returns the result in an HTML table.
- Explanation: When the query is sent from the JavaScript to the PHP file, the following happens:
 - PHP opens a connection to a MySQL server
 - The correct person is found
 - An HTML table is created, filled with data, and sent back to the "txtHint" placeholder

model-view-controller (MVC)

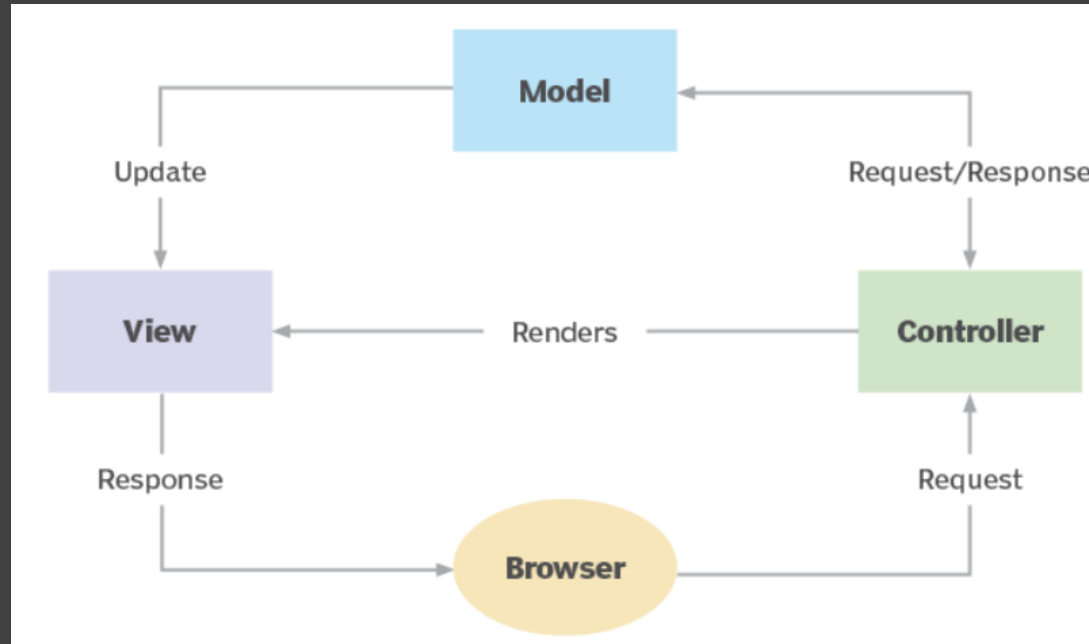
- The Model-View-Controller (MVC) framework is an architectural/design pattern that separates an application into three main logical components Model, View, and Controller.
- It organizes an application's logic into distinct layers, each of which carries out a specific set of tasks.
- It isolates the business logic and presentation layer from each other.
- The MVC methodology incorporates the entire application, from the user interface (UI) to the underlying data model.
- It was traditionally used for desktop graphical user interfaces (GUIs). Nowadays, MVC is one of the most frequently used industry-standard web development frameworks to create scalable and extensible projects. It is also used for designing mobile apps.

MVC (cont...)

- MVC was created by Trygve Reenskaug. The main goal of this design pattern was to solve the problem of users controlling a large and complex data set by splitting a large application into specific sections that all have their own purpose.
- Features of MVC :
 - It provides a clear separation of business logic, UI logic, and input logic.
 - It offers full control over your HTML and URLs which makes it easy to design web application architecture.
 - It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.
 - It supports Test Driven Development (TDD).

MVC (cont...)

- Components of MVC :
 - The MVC framework includes the following 3 components:
 - Controller
 - Model
 - View



- Fig: Conceptualize the MVC application architectural design pattern model.

MVC (cont...)

MVC design layers

- The MVC methodology separates an application's logic into three distinct layers:
 - Model.
 - The model layer is responsible for the application's data logic and storing and retrieving data from back-end data stores.
 - The model layer might also include mechanisms for validating data and carrying out other data-related tasks. This layer is responsible for maintaining all aspects of the data and ensuring its integrity and accessibility.
 - View.
 - The view layer provides the UI necessary to interact with the application.
 - It includes components needed to display the data and enables users to interact with that data. For example, the view layer might include buttons, links, tables, drop-down lists or text boxes.
 - Controller.
 - The controller layer contains the application logic necessary to facilitate communications across the application, acting as an interface between the view and model layers.
 - The controller is sometimes viewed as the brains of the application, keeping everything moving and in sync.

MVC (cont...)

The MVC architectural pattern allows us to adhere to the following design principles:

1. Divide and conquer. The three components can be somewhat independently designed.
2. Increase cohesion. The components have stronger layer cohesion than if the view and controller were together in a single UI layer.
3. Reduce coupling. The communication channels between the three components are minimal and easy to find.
4. Increase reuse. The view and controller normally make extensive use of reusable components for various kinds of UI controls. The UI, however, will become application-specific, therefore it will not be easily reusable.
5. Design for flexibility. It is usually quite easy to change the UI by changing the view, the controller, or both.

MVC (cont...)

Advantages of MVC

1. Codes are easy to maintain and they can be extended easily.
2. The MVC model component can be tested separately.
3. The components of MVC can be developed simultaneously.
4. It reduces complexity by dividing an application into three units. Model, view, and controller.
5. It supports Test Driven Development (TDD).
6. It works well for Web apps that are supported by large teams of web designers and developers.
7. This architecture helps to test components independently as all classes and objects are independent of each other
8. Search Engine Optimization (SEO) Friendly.

MVC (cont...)

Disadvantages of MVC

1. It is difficult to read, change, test, and reuse this model
2. It is not suitable for building small applications.
3. The inefficiency of data access in view.
4. The framework navigation can be complex as it introduces new layers of abstraction which requires users to adapt to the decomposition criteria of MVC.
5. Increased complexity and Inefficiency of data

Popular MVC Frameworks:

Some of the most popular and extensively used MVC frameworks are listed below.

- Ruby on Rails
- Django
- CherryPy
- Spring MVC
- Catalyst
- Rails
- Zend Framework
- Fuel PHP
- Laravel
- Symphony